



Unit 5

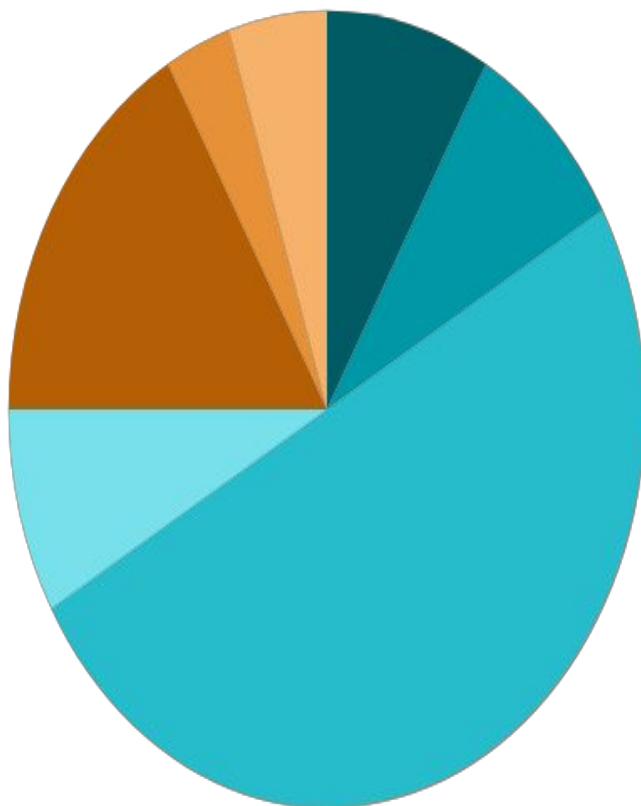
Big Data Tools



Outline

- Hbase – data model and implementations
- Hbase clients
- Hbase examples, praxis.
- Cassandra – Cassandra data model
- Cassandra examples
- cassandra clients.
- Pig – Grunt – pig data model
- Pig Latin – developing and testing Pig Latin scripts.
- Hive – data types and file formats
- HiveQL data definition
- HiveQL data manipulation – HiveQL queries

Lecture Structure



	05 Minutes	Outline & Objectives
	05 Minutes	Recap of Previous Lecture
	30 Minutes	Delivery of Actual Content as per Lesson Plan
	05 Minutes	Industry Relevance
	10 Minutes	Interactive Activities
	02 Minutes	Importance of Discipline
	03 Minutes	Recording of Attendance

HBase?

What is HBase?

- Open-source, column-oriented **distributed database system**.
- Built to work in the **Hadoop ecosystem**.
- Inspired by **Google's Bigtable**, written in **Java**.
- Designed for **real-time Big Data applications**.

Key Capabilities

- Handles **terabytes to petabytes** of data.
- Tables consist of **billions of rows** and **millions of columns**.
- Optimized for **low latency** operations.

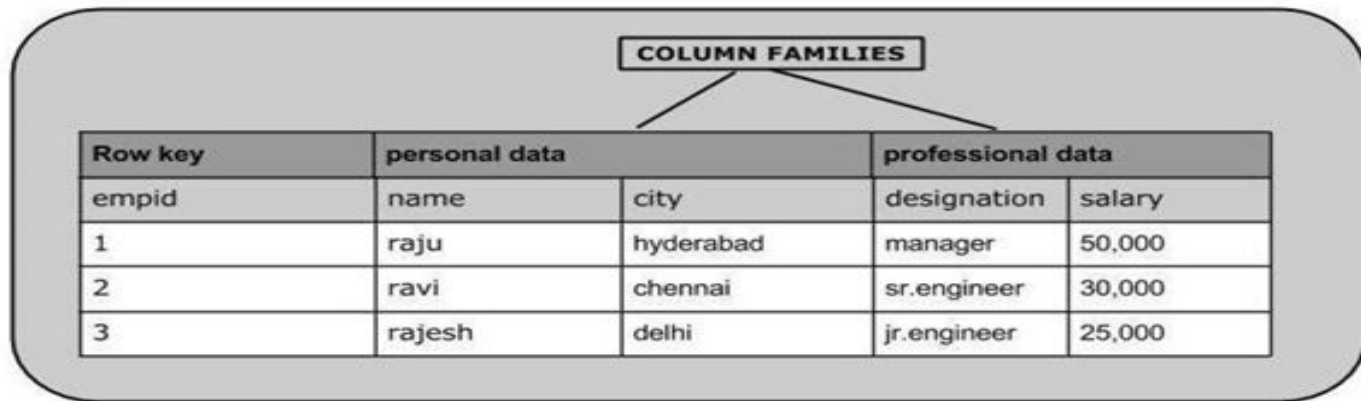
Unique Features of HBase

- ◆ Built for **low-latency** operations
- ◆ Supports **random read/write** access
- ◆ Stores data in **key/value pairs** (columnar storage)
- ◆ **Automatic & configurable sharding** of tables
- ◆ Efficient management of **massive table structures**



The following image shows column families in a column-oriented database:

Rowid	Column Family			Column Family			Column Family			Column Family		
	col1	col2	col3	col1	col2	col3	col1	col2	col3	col1	col2	col3
1												
2												





Customer ID	Name	Address	Product ID	Product Name
1	Paul Walker	US	231	Gallardo
2	Vin Diesel	Brazil	520	Mustang

If this table is stored in a **row-oriented database**. It will store the records as shown below:

1, Paul Walker, US, 231, Gallardo, 2, Vin Diesel, Brazil, 520, Mustang

While the **column-oriented databases** store this data as:

1,2, Paul Walker, Vin Diesel, US, Brazil, 231, 520, Gallardo, Mustang



Hbase VS RDBMS

HBase	RDBMS
Schema-less database	Fixed schema database
Column-oriented	Row-oriented
Designed for denormalized data	Designed for normalized data
Supports wide and sparse tables	Tables are typically thin
Automatic partitioning support	No built-in partitioning
Best for OLAP (Analytics)	Best for OLTP (Transactions)
Reads only relevant data	Retrieves entire rows , even if not needed
Handles structured & semi-structured data	Handles structured data only
Efficient aggregation over rows/columns	Aggregation is expensive

Logical Components of HBase Data Model

Component	Description
Table	Logical collection of rows, divided into Regions , each served by one Region Server .
Row	A single record identified by a unique rowkey (always a byte[]).
Column Family	Group of related columns stored together in HFiles .
Column	Defined as columnfamily:columnname; columns within a family can vary across rows.
Cell	Intersection of rowkey, column family, and column; stores value as byte[].
Version	Each cell holds multiple versions of data, identified by timestamp . Default: 3 versions.

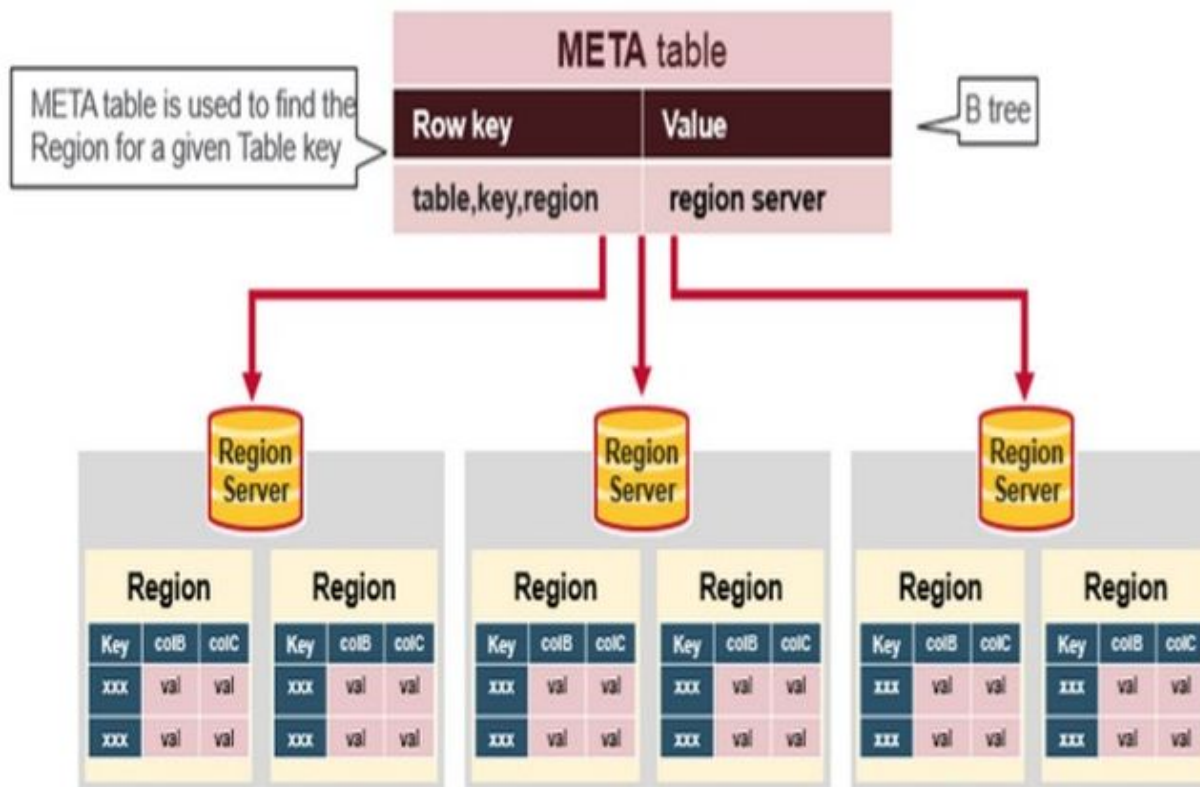


Hbase Example

Row Key	Customer		Sales	
Customer Id	Name	City	Product	Amount
101	John White	Los Angeles, CA	Chairs	\$400.00
102	Jane Brown	Atlanta, GA	Lamps	\$200.00
103	Bill Green	Pittsburgh, PA	Desk	\$500.00
104	Jack Black	St. Louis, MO	Bed	\$1600.00

Column Families

HBASE





HBase Clients

HBase supports multiple client interfaces:

- HBase Shell (CLI)
- Kundera (Java Object Mapper)
- REST Client (Web-based APIs)
- Thrift Client (Cross-language RPC)

HBase Shell (CLI Tool)

- Command-line interface for interacting with HBase

- Start shell:

```
$ ./bin/hbase shell
```

- **Create Table**

```
create 'emp', 'personal data', 'professional data'
```

- **Insert Data**

```
put 'emp','1','personal data:name','raju'
```

- **Read Data**

```
get 'emp', '1'
```

- **Update Cell**

•

```
put 'emp','1','personal data:city','Delhi'
```

- **Delete Cell**

```
delete 'emp','1','personal data:name'
```

Kundera – Object Mapper (Java)

- JPA 2.1-compliant object mapper for NoSQL and RDBMS
- Supports 8 NoSQL databases
- Maps Java objects to HBase data using annotations
- Features:
 - Easy object-relational mapping
 - Secondary caching
 - Robust querying

CRUD Operations with Kundera

- **Employee Entity:**
- @Entity
- @Table(name = "EMPLOYEE")
- public class Employee {
- @Id private String employeeId;
- @Column private String employeeName;
- @Column private String address;
- @Column private String department;
- }

CRUD Example:

- Create: em.persist(employee)
- Read: em.find(Employee.class, "1")
- Update: em.merge(foundEmployee)
- Delete: em.remove(foundEmployee)



REST Client

- Allows interaction with HBase over HTTP
- Data is returned in structured formats (JSON, XML)
- REST modules handle parts of transactions
- Common methods:
 - GET (Read)
 - POST (Create)
 - PUT (Update)
 - DELETE (Delete)

Use case example:

- Apps like BookMyShow use REST APIs to fetch updated data dynamically



Thrift Client

- RPC-based framework developed by Facebook
- Cross-language client-server communication
- Uses **.thrift** files to define:
 - Data types
 - Service interfaces
- Supports multiple programming languages
- Ideal for complex distributed systems

Java API – HBase Table Creation Example

Use Case: Import data from Data_file.txt

Table Structure:

- Column Family: **region**
 - Columns: country, state, city
- Column Family: **time**
 - Columns: year, month

Required JARs:

- commons-logging-1.0.4
- hadoop-core-0.20.2-cdh3u2
- hbase-0.90.4-cdh3u2
- log4j-1.2.15
- zookeeper-3.3.3-cdh3u0

Cassandra

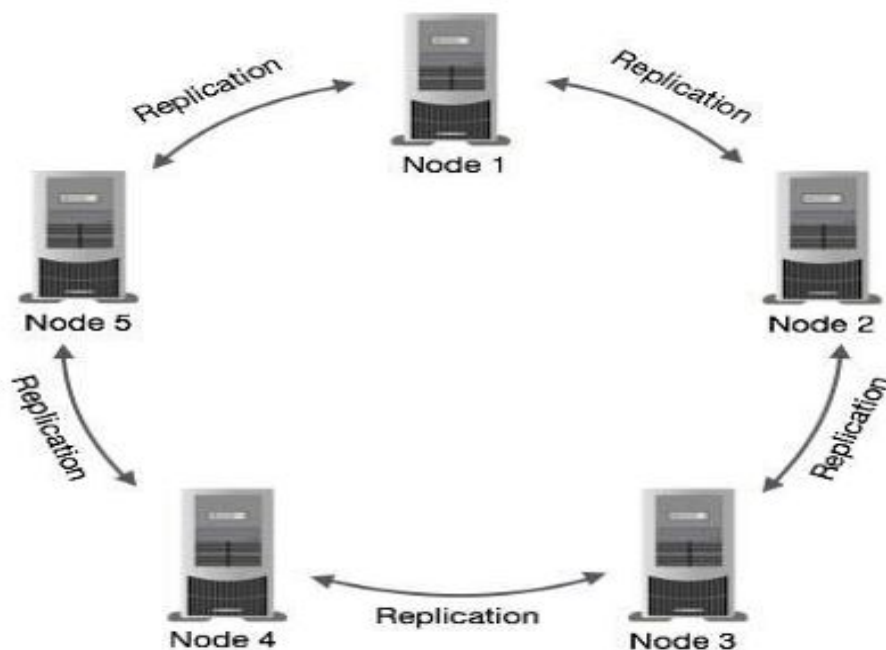
Introduction

- Distributed database management system
- Handles large volumes of structured data
- Built for high availability with no single point of failure
- Column-oriented database model

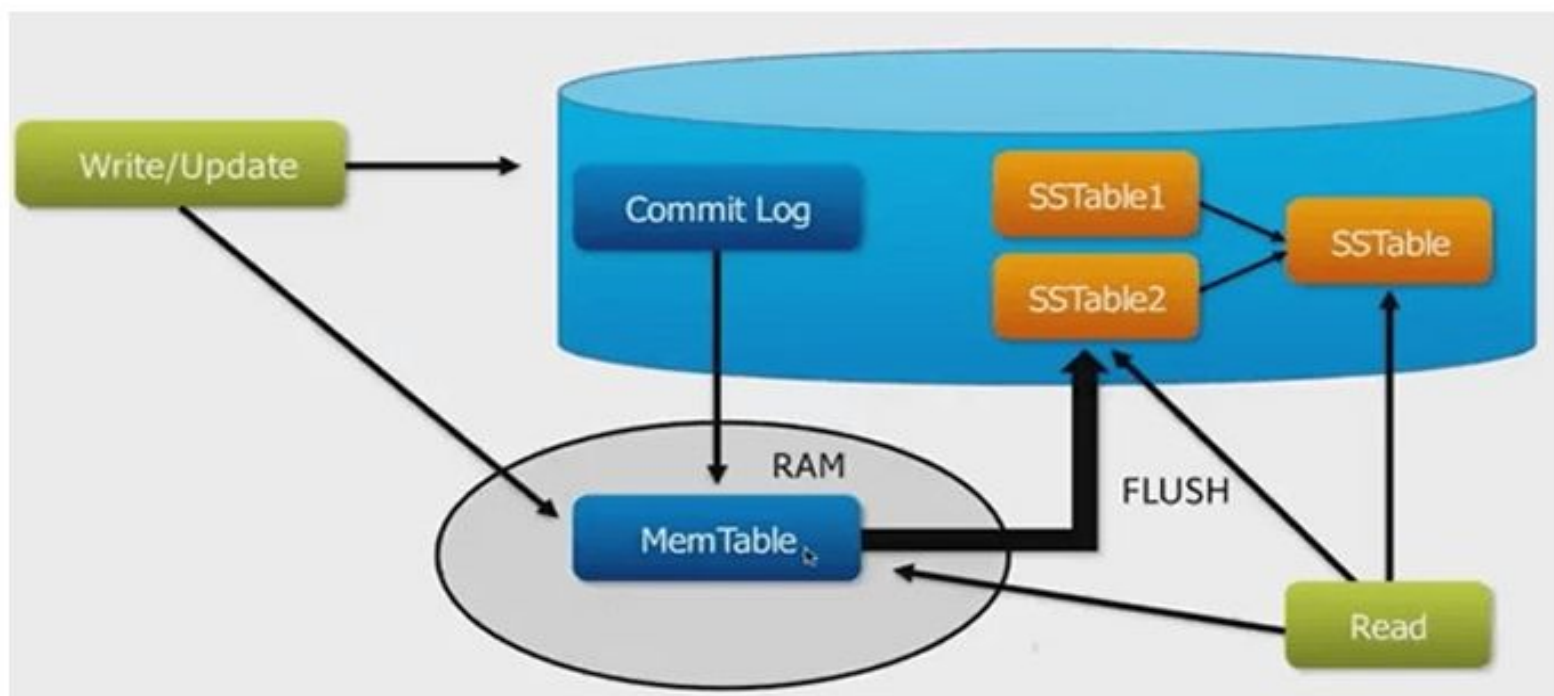
Key Features

- **Massively Scalable Architecture**
Masterless design enables easy scale-out
- **Masterless Architecture**
Read/write from any node
- **Linear Scale Performance**
Performance improves with more nodes
- **No Single Point of Failure**
Data replicated across nodes
- **Fault Detection & Recovery**
Easy node recovery
- **Flexible Data Model**
Fast reads and writes
- **Data Compression**
Up to 80% without performance loss

Apache Cassandra's peer-to-peer architecture



Cassandra Architecture



Cassandra Architecture

- Peer-to-peer distributed system
- Nodes are independent yet interconnected
- Any node can accept read/write requests
- High fault tolerance: failure of one node doesn't impact service

Components of Cassandra

1. Cluster

- A **group of nodes (computers)** working together.
- All nodes are equal—there's **no master-slave setup**.

2. Node

- A **single machine** in the cluster.
- Each node stores part of the data.

3. Data Center

- A **collection of nodes**, often grouped by location (like different offices or cloud regions).

Cassandra Architecture

4. Commit Log

- Every write is first saved here.
- Helps in **data recovery** if the system crashes.

5. MemTable

- An **in-memory** data store.
- After data is written to the commit log, it's saved here temporarily.

6. SSTable (Sorted String Table)

- When MemTable gets full, data is written to disk as an **SSTable**.
- These are **immutable** (can't be changed once written).

7. Bloom Filter

- A quick-check system to see if data might exist in an SSTable.
- Helps avoid unnecessary reads.

Use Cases

- High availability applications
- Big data analytics
- Real-time recommendation engines
- Internet of Things (IoT) systems

Companies Using Cassandra

- Facebook , Twitter, Netflix ,Cisco,eBay. Rackspace

Cassandra Data Model

Overview

- The data model defines how data is stored in Cassandra.
- Distributed across multiple machines in a **ring** formation.
- **Cluster** is the top-level container, consisting of multiple nodes.

Cluster in Cassandra

- Collection of nodes forming a ring-like structure.
- Data is **partitioned and replicated** across nodes.
- **Replication ensures fault tolerance**—if one node fails, another can serve the data.

Cassandra Data Model Components

Cassandra stores data in the following hierarchy:

1. **Keyspaces**
2. **Tables (Column Families)**
3. **Columns**

Cassandra Data Model Components

Keyspaces

- Highest-level data structure, similar to a database or schema in RDBMS.
- **Must be created before tables.**
- One keyspace can have **multiple tables**.
- **Replication settings** are defined at the keyspace level.
 - e.g., Replication factor = 3 → 3 copies of each data row

Tables (Column Families)

- Defined **within a keyspace**.
- Contain **rows** and **columns**.
- Each table has a **primary key**.
- Tables are where actual data is stored.

Columns

- Define the structure and data types in a table.
- Each column has a name and a type (e.g., int, text, boolean, etc.)
- Flexible schema: You can have different columns per row if needed.

Creating a Keyspace

Example:

```
CREATE KEYSPACE myapp WITH REPLICATION = {  
'class' : 'SimpleStrategy', 'replication_factor' : 1};
```

- Use SimpleStrategy for dev/testing.
- Use NetworkTopologyStrategy for production

- **Creating a Table**

```
CREATE TABLE users (user_id int PRIMARY  
KEY, name TEXT, email TEXT, age INT);
```

- **Inserting Data:**

```
INSERT INTO users (user_id, name, email, age)  
VALUES (1, 'Alice', 'alice@example.com',  
30);
```

- **Querying Data:**

```
SELECT * FROM users;
```



- **Updating Data:**

UPDATE users SET age = 31, email =
'alice.new@example.com' WHERE user_id = 1;

- **Deleting Data**

DELETE FROM users WHERE user_id = 1;

Cassandra Client

- Client drivers let applications talk to Cassandra.
- Drivers are organized by programming language.
- Always check:
 - Cassandra **version compatibility**
 - **Features supported** by the driver

Java Client Drivers

- Achilles
- Astyanax
- Kundera
- PlayORM

Python Client Drivers

- Datastax Python driver

Ruby Client Drivers

- Datastax Ruby driver

Cassandra Client

C# / .NET Drivers

- Cassandra Sharp
- Datastax C# driver
- Fluent Cassandra

PHP Client Drivers

- CQL | PHP
- Datastax PHP driver
- PHP-Cassandra
- PHP Library for Cassandra

Cassandra + Hadoop Integration

1. Set Same Partitioner in cassandra.yaml

partitioner: org.apache.cassandra.dht.Murmur3Partitioner

2. Configure Hadoop in core-site.xml

<property>

<name>fs.defaultFS</name>

<value>hdfs://namenode:9000</value>

</property>

3. Add Hadoop-Cassandra Connector

- Use **Datastax Hadoop Connector** or **Spark-Cassandra Connector**
- Include JARs in your classpath or Maven dependencies

4. Sample MapReduce Code (Java)

```
job.setInputFormatClass(CqlInputFormat.class);  
  
ConfigHelper.setInputInitialAddress(conf,  
    "127.0.0.1");  
  
ConfigHelper.setInputPartitioner(conf,  
    "Murmur3Partitioner");  
  
ConfigHelper.setInputColumnFamily(conf,  
    "mykeyspace", "mytable");
```


HIVE

Hadoop Ecosystem Tools

- **Sqoop**: Transfers data between HDFS and RDBMS.
- **Pig**: Uses scripting language for MapReduce operations.
- **Hive**: Uses SQL-like queries (HiveQL) for data processing.

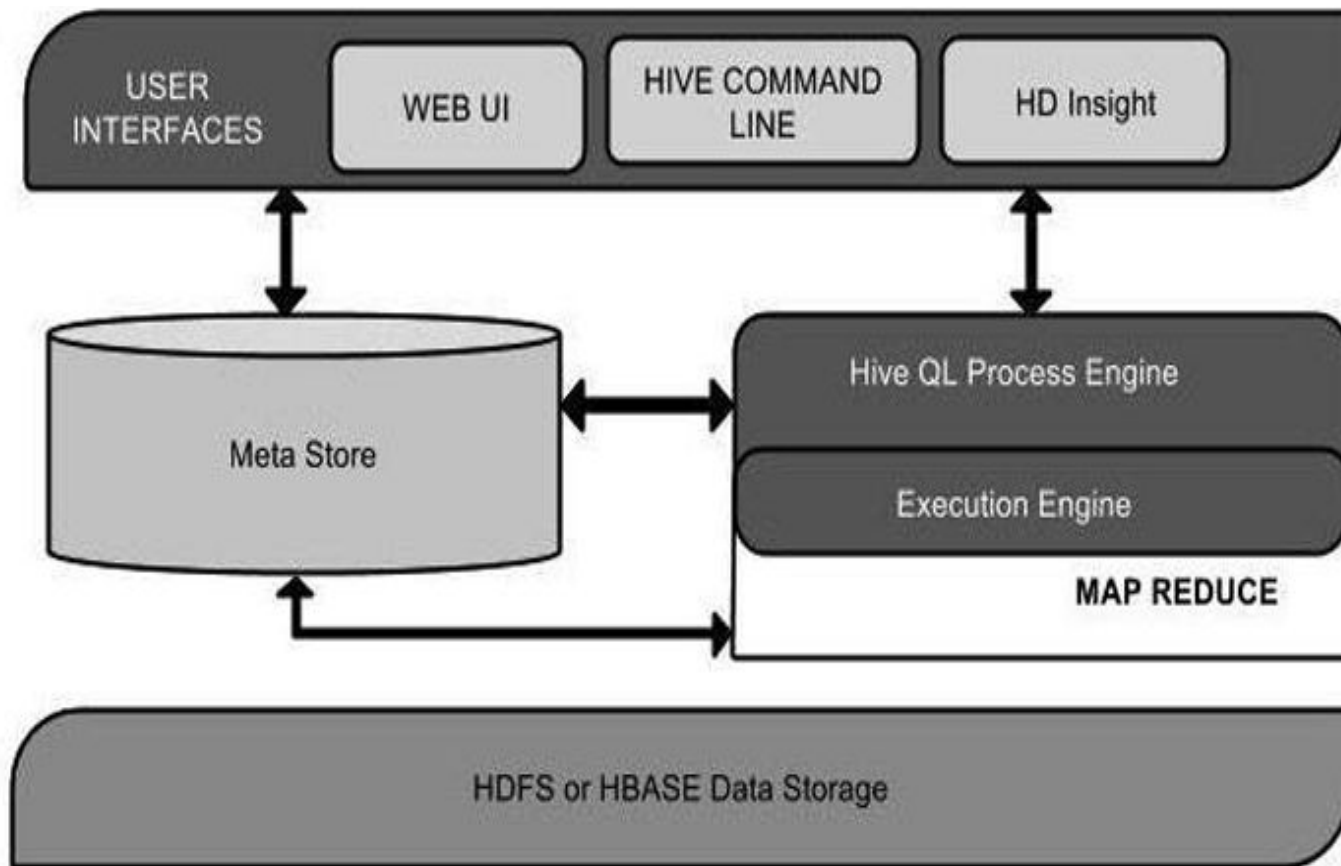
What is Hive?

- Data warehouse tool built on top of Hadoop.
- Simplifies querying & analyzing structured data using HiveQL.
- Initially developed by Facebook; now maintained by Apache.

Features of Hive

- Stores **schema** in a database & **data** in HDFS.
- Designed for **OLAP** (Online Analytical Processing).
- Uses **HiveQL**, a SQL-like language.
- **Familiar, fast, scalable, and extensible.**

Architecture of Hive



Architecture of Hive

1. User Interface

- Interfaces for user interaction with HDFS via Hive.
- **Supported UI tools:**
 - Hive Web UI
 - Hive Command Line

Hive HDInsight (Windows)

2. Meta Store

- Stores **schema/metadata** of:
 - Databases
 - Tables
 - Columns and data types
 - HDFS mappings

Uses **traditional RDBMS** for storage.

3. HiveQL Process Engine

- Interprets and compiles **HiveQL** (SQL-like queries).
- Converts queries into **MapReduce jobs**.
- Alternative to writing Java-based MapReduce code.

Architecture of Hive

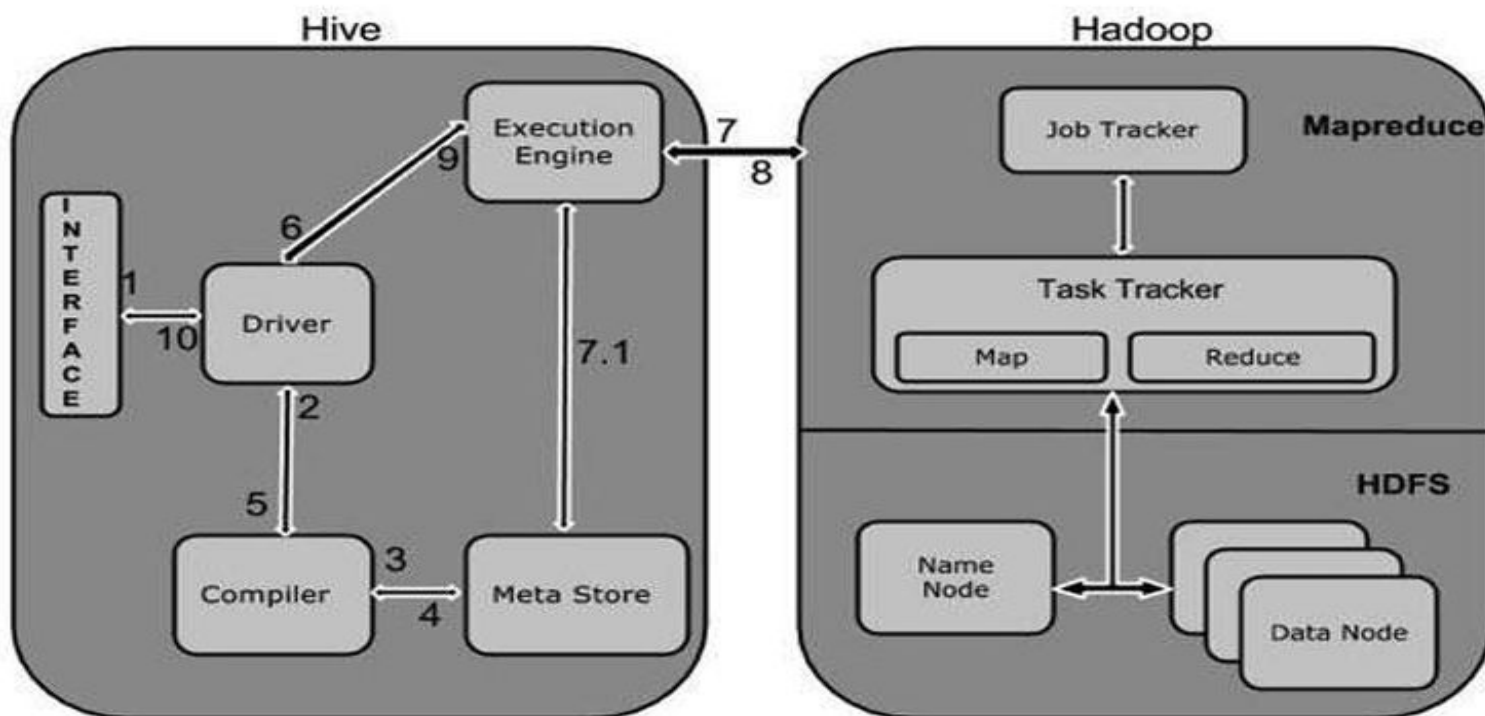
4. Execution Engine

- Connects HiveQL Process Engine with **MapReduce**.
- Processes and executes queries.
- Returns results equivalent to MapReduce outputs.

5. HDFS or HBase

- **Data storage layer.**
- Stores the actual data in either:
 - Hadoop Distributed File System (**HDFS**)
 - **HBase** (NoSQL)

Working of Hive



Working of Hive

Step-by-Step Process

1. **Execute Query**
User sends query via Hive CLI/Web UI → Driver (JDBC/ODBC).
2. **Get Plan**
Driver sends query to **Compiler** to parse & create query plan.
3. **Get Metadata**
Compiler requests metadata from **Metastore**.
4. **Send Metadata**
Metastore responds with metadata.
5. **Send Plan**
Compiler finalizes plan → sends it to Driver.
6. **Execute Plan**
Driver forwards plan to **Execution Engine**.
7. **Execute Job**
Execution Engine starts a **MapReduce job**:
 - Sent to **JobTracker** (NameNode)
 - Assigned to **TaskTracker** (DataNode)
 - **7.1 Metadata Ops**: May query **Metastore** again if needed.
8. **Fetch Result**
Execution Engine collects results from DataNodes.
9. **Send Results**
Execution Engine → Driver.
10. **Return to User**
Driver returns final output to Hive Interface.

Hive Data Types

Column Types :Integral Types

Type	Postfix	Example
TINYINT	Y	10Y
SMALLINT	S	10S
INT		10
BIGINT	L	10L

String Types

- VARCHAR(1–65355)
- CHAR(255)
- Use ' ' or " " for string literals
- Supports C-style escape characters

Timestamps & Dates

- Timestamp format:
YYYY-MM-DD HH:MM:SS.fffffffff
- DATE: YYYY-MM-DD

•Decimal & Union Types

- DECIMAL(precision, scale) — e.g., decimal(10,0)
- UNIONTYPE<int, double, array<string>, struct<a:int,b:string>>

Hive Data Types

Literals

- **Floating Point:** Use DOUBLE
- **Decimal Type:** Higher precision than DOUBLE
(range $\sim -10^{308}$ to 10^{308})

Null Values

- Represented by: NULL

Complex Types

- **ARRAY<data_type>**
→ Example: ARRAY<STRING>
- **MAP<primitive_type, data_type>**
→ Example: MAP<STRING, INT>
- **STRUCT<col_name:data_type [COMMENT]>**
→ Example: STRUCT<name:STRING, age:INT>

1. Text File (Default)

- Simple, plain-text format
- Records stored line-by-line, newline-terminated (\n)
- Supports compression (e.g., BZIP2)
- Easy data interchange
- **Syntax:** CREATE TABLE table_name (...) STORED AS TEXTFILE;



2. Sequence File

- Hadoop flat files storing binary key-value pairs
- Splittable and efficient for merging files
- **Syntax:** CREATE TABLE table_name (...) STORED AS SEQUENCEFILE;



3. RC File (Row Columnar)

- Combines row and columnar storage
- High row-level compression
- Great for multiple row-level operations
- **Syntax:**
CREATE TABLE table_name (...) STORED AS RCFILE;

Hive File Formats



4. AVRO File

- Schema stored in JSON
- Data stored in compact binary format
- Easy integration with external systems
- **Syntax:** CREATE TABLE table_name (...) STORED AS AVRO;



5. ORC File (Optimized Row Columnar)

- Highly efficient for large Hive tables
- Improves read/write/processing performance
- Designed to overcome limits of earlier formats
- **Syntax:** CREATE TABLE table_name (...) STORED AS ORC;



6. Parquet File

- Column-oriented binary format
- Ideal for analytic workloads
- Efficient column-wise compression (Snappy, Gzip)
- **Syntax:** CREATE TABLE table_name (...) STORED AS PARQUET;

HiveQL Data Definition

Command	Used With
CREATE	Database, Table
SHOW	Databases, Tables, Functions
DESCRIBE	Databases, Tables
USE	Database
DROP	Database, Table
ALTER	Database, Table
TRUNCATE	Table

CREATE DATABASE

Syntax: CREATE DATABASE IF NOT EXISTS bda;

SHOW DATABASES

Syntax: SHOW DATABASES;

USE DATABASE

Syntax: USE bda;

DESCRIBE DATABASE

Syntax: DESCRIBE DATABASE bda;

DROP DATABASE

Syntax: DROP DATABASE IF EXISTS bda CASCADE;

- **CREATE TABLE**

```
CREATE TABLE IF NOT EXISTS Employee(Emp_ID  
STRING COMMENT 'Employee ID',  
Emp_designation STRING COMMENT  
'Designation');
```

- **SHOW TABLES**

Syntax: SHOW TABLES;

- **DESCRIBE TABLE**

Syntax: DESCRIBE Employee;

- **DROP TABLE:**

- Syntax: DROP TABLE IF EXISTS Employee PURGE;

- **ALTER TABLE**

ALTER TABLE Employee RENAME TO Comp_Emp;

Add Columns: ALTER TABLE Comp_Emp ADD COLUMNS
(emp_dob STRING, emp_contact STRING);

- **TRUNCATE TABLE**

Syntax: TRUNCATE TABLE Comp_Emp;

Hive DML Commands

1. LOAD

- Moves data files into Hive tables.

- **Syntax:**

```
LOAD DATA [LOCAL] INPATH 'filepath' [OVERWRITE] INTO TABLE tablename [PARTITION (...)]
```

- **Example:**

```
LOAD DATA LOCAL INPATH '/home/user/dab' INTO TABLE emp_data;
```

2. SELECT

- Retrieves data from tables.

- **Example:**

```
SELECT * FROM emp_data;
```

3. INSERT

- **INSERT INTO:** Appends data

- **INSERT OVERWRITE:** Replaces data

- **INSERT ... VALUES:** Direct row insertion

- **Example:**

```
INSERT INTO TABLE student VALUES(101, 'Callen', 'IT', '7.8');
```

Hive DML Commands

4. DELETE

- Deletes rows based on condition.

- Example:**

```
DELETE FROM student WHERE roll_no=105;
```

5. UPDATE

- Updates column values conditionally.

- Example:**

```
UPDATE student SET branch='IT' WHERE roll_no=103;
```

6. EXPORT

- Exports table data to HDFS.

- Example:**

```
EXPORT TABLE student TO 'export_from_hive';
```

7. IMPORT

- Imports data into a Hive table.

- Example:**

```
IMPORT TABLE imported_table FROM 'export_from_hive';
```


Pig

What is Pig?

- High-level data flow platform for executing MapReduce programs.
- Developed by Yahoo; language used: **Pig Latin**.
- Scripts internally convert to MapReduce jobs and execute on **HDFS**.
- Handles **structured**, **semi-structured**, and **unstructured** data.

Features of Apache Pig:

- Rich set of operators (join, sort, filter, etc.).
- Easy programming (similar to SQL).
- Automatic execution optimization.
- Extensibility through custom functions.
- Support for **User Defined Functions (UDFs)** in Java.
- Analyzes all types of data and stores results in HDFS.

Apache Pig

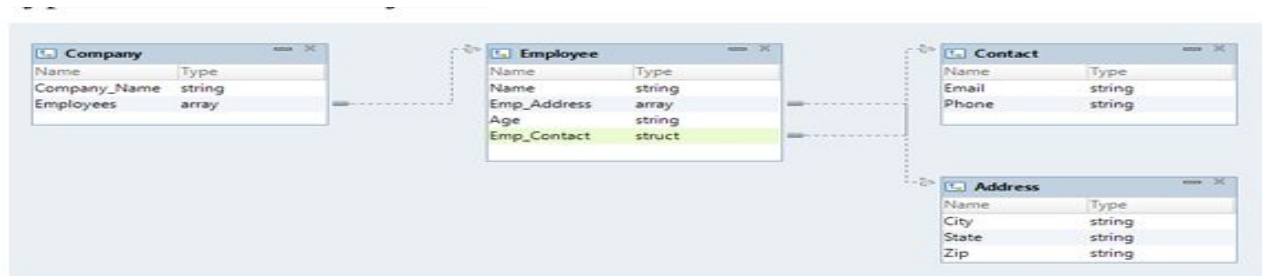
Advantages of Apache Pig:

- **Less Code:** Fewer lines needed compared to Java MapReduce.
- **Reusability:** Scripts are flexible and reusable.
- **Nested Data Types:**
 - Supports complex types like **Tuple**, **Bag**, and **Map**.

Example: Nested Data Type - Company

yaml

```
Company: { name: chararray, employees: bag
  { employee: tuple (id: int, name: chararray, salary: float) } }
```



Apache Pig Architecture

Overview:

- Pig Latin: High-level data processing language with rich types and operators.
- Scripts executed through: **Grunt Shell**, **UDFs**, or **Embedded** methods.
- Internally converted into **MapReduce** jobs for execution.

Apache Pig Components:

1. Parser:

1. Syntax and type checking.
2. Generates a **DAG (Directed Acyclic Graph)** of
3. logical operations.

2. Optimizer:

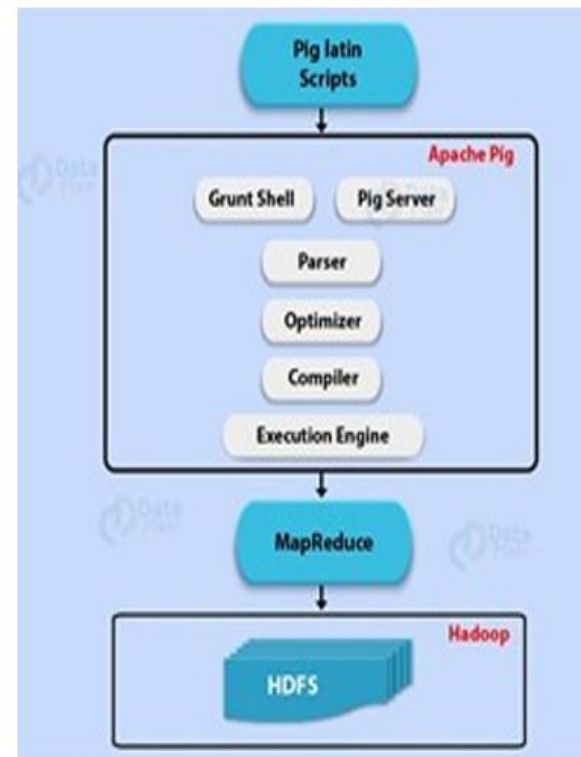
1. Optimizes the logical plan (e.g., **projection**, **push**

3. Compiler:

1. Converts logical plan into **MapReduce jobs**.

4. Execution Engine:

1. Submits and runs MapReduce jobs on **Hadoop**,
2. producing the results



Pig Latin Data Model

Data Types in Pig Latin:

Data Type	Description	Example
Atom	Single value (string/number)	'raja', 30
Tuple	Ordered set of fields	(Raja, 30)
Bag	Collection of tuples	{(Raja, 30), (Mohammad, 45)}
Map	Key-value pairs	[name#Raja, age#30]
Relation	Bag of tuples (similar to table)	{(Raja, 30), (Ali, 25)}

Grunt Shell in Apache Pig

What is Grunt?

- A shell where Pig Latin scripts are executed.
- Start Grunt Shell in:
 - **Local mode:** \$./pig -x local
 - **MapReduce mode:** \$./pig -x mapreduce
- Exit using Ctrl + D or quit command.

Writing and Running Scripts:

- Direct entry of Pig Latin commands.
- Example:
grunt> customers = LOAD 'customers.txt' USING PigStorage(',');

Shell Commands:

sh: Execute Linux shell commands from Grunt.

Example: grunt> sh ls

fs: Execute HDFS file system commands. Example: grunt> fs -ls

Utility Commands in Grunt Shell

Utility Commands Overview:

- **clear**: Clears the screen.
 - **Example**: `grunt> clear`
- **help**: Lists Pig commands/properties.
 - **Example**: `grunt> help`
- **history**: Shows previously executed commands.
 - **Example**: `grunt> history`
- **set**: Sets job properties.
 - **Example**: `grunt> set job.priority high`
- **quit**: Exits the Grunt Shell.

Execution Commands:

- **exec**: Executes a Pig script file.
 - **Example**: `grunt> exec /sample_script.pig`
- **kill**: Terminates a running Pig job.
 - **Example**: `grunt> kill JobId`

Introduction to Pig Latin

- High-level data flow language for processing large datasets.
- Similar to SQL, but more powerful for Hadoop.
- Supports **complex data types, operators, and relational operations**

Simple Types:

- **Int:** 32-bit Integer (Ex: 8)
- **Long:** 64-bit Integer (Ex: 5L)
- **Float:** 32-bit Floating Point (Ex: 5.5F)
- **Double:** 64-bit Floating Point (Ex: 10.5)
- **Chararray:** UTF-8 String (Ex: 'tutorials point')
- **Bytearray:** Binary Data (Blob)
- **Boolean:** true/false
- **Datetime:** Date-Time (Ex: 1970-01-01T00:00:00.000+00:00)

Complex Types:

- **BigInteger:** Very large integers (Ex: 60708090709)
- **BigDecimal:** High precision decimals (Ex: 185.98376256272893883)
- **Tuple:** Ordered fields (Ex: (raja, 30))
- **Bag:** Collection of tuples (Ex: {(raju,30), (Mohhammad,45)})

Map: Key-value pairs (Ex: ['name'#'Raju','age'#30])

Pig Latin – Arithmetic Operators

Operator	Description	Example
+	Addition	$a+b = 30$
-	Subtraction	$a-b = -10$
*	Multiplication	$a*b = 200$
/	Division	$b/a = 2$
%	Modulus (remainder)	$b\%a = 0$

Conditional Operators

- **Bincond:** Ternary Operator ((condition) ? true_val : false_val)
- **CASE WHEN THEN ELSE END:** Like SQL switch case
- Comparison Operators



Pig Latin – Relational Operations

- **LOAD:** Load data
- **STORE:** Save data
- **FILTER:** Remove unwanted rows
- **DISTINCT:** Remove duplicates
- **JOIN:** Join relations
- **GROUP:** Group by fields
- **ORDER:** Sort records
- **LIMIT:** Limit output

Sample Script: Employee_details

```
Employee = LOAD  
'hdfs://localhost:9000/pig_data/Employee_details.txt'  
USING PigStorage(',')  
AS (id:int, firstname:chararray, lastname:chararray,  
phone:chararray, city:chararray);  
  
Employee_order = ORDER Employee BY age DESC;  
Employee_limit = LIMIT Employee_order 4;  
DUMP Employee_limit;
```

How to Execute a Pig Script

- **Modes:** Local Mode / MapReduce Mode
- **Shell Execution:** Use `grunt>` prompt
- **Script Execution:** `pig sample_script.pig`

Testing Pig Scripts – PigUnit

- Unit Testing for Pig Scripts
- Based on **JUnit**
- Local Mode (No cluster needed)
- Supports unit testing, regression testing, prototyping

Setting Up for PigUnit

1. Install Maven Plugin for Eclipse (M2Eclipse)
2. Install Pig Eclipse Plugin (pigEditor)
3. Create Maven Project
4. Add Dependencies (Pig, PigUnit)

PigUnit Word Count Example

Pig Script (wordcount.pig):

```
A = load '/home/kiran/workspace/pigunit/input.data';
```

```
B = foreach A generate flatten(TOKENIZE((chararray)$0)) as word;
```

```
C = group B by word;
```

```
D = foreach C generate COUNT(B), group;
```

```
dump D;
```

PigUnit Java Test

```
public void testwordcount() throws Exception {
```

```
    PigTest pigTest = new PigTest("/path/wordcount.pig");
```

```
    String[] output = {"(1,all)", "(1,from)", "(1,Hello)", "(1,Brindavan!)"};
```

```
    pigTest.assertOutput("D", output);
```

```
}
```

Tokenize in Pig Latin

USA USA EUROPE EUROPE EUROPE
EUROPE USA USA USA EUROPE EUROPE USA
EUROPE USA

- **Pig Script:**

```
inp = LOAD '/user/countries.txt' AS (singleline);  
tknz = FOREACH inp GENERATE  
TOKENIZE(singleline) AS Words;
```